

CX016-2.5-3-IML & Introduction to Machine Learning

Introduction to Python for Data Science

TOPIC LEARNING OUTCOMES

At the end of this topic, you should be able to:

1. Understanding Python Basics and for Data Science.
2. Data Manipulation and Analysis.
3. Implementing Basic Data Science Techniques

Contents & Structure

Introduction to Python Programming
Working with Data Structures
Introductions to Data Science libraries
Data Manipulation with pandas
Data Visualization with Matplotlib and Seaborn
Introduction to Basic Statistical Analysis
Data Pre-processing
Scikit-Learn library and its functions

Recap From Last Lesson

- What are the major types of ML?
- What are the applications of ML?
- What are the benefits of predictive ML models?



Installation Guides



- Python is a programming language that lets you work more quickly and integrate your systems more effectively.
- Key Features:
 - Readable and Simple Syntax: Python's syntax is clear and easy to understand.
 - Interpreted Language: Python code is executed line-by-line, which makes debugging easier.
 - Dynamically Typed: You don't need to declare the type of variable; it is inferred at runtime.
 - Extensive Standard Library: Python comes with a vast standard library.
 - Cross-Platform: Python can run on various operating systems.
 - Open Source: Python is free to use and distribute, even for commercial purposes.



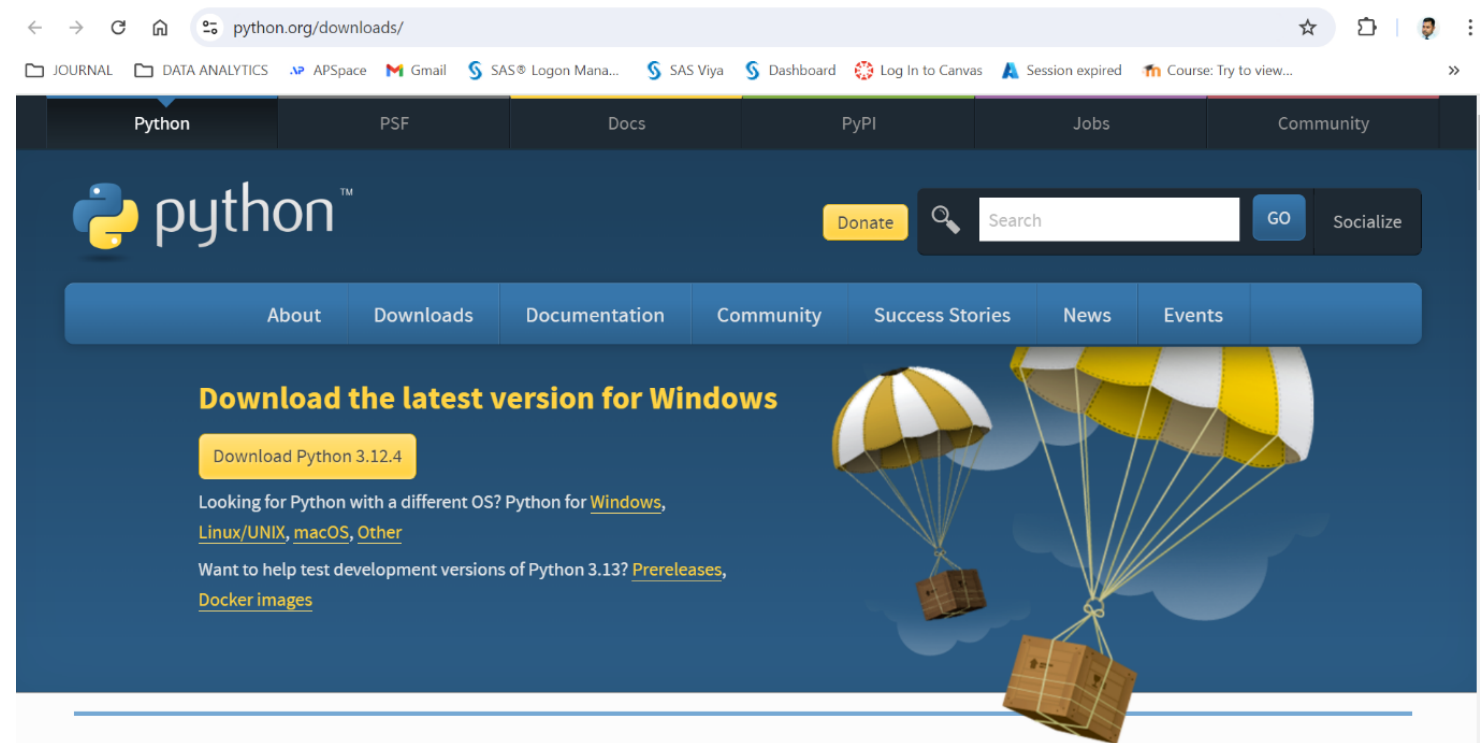
- Python is a programming language that lets you work more quickly and integrate your systems more effectively.
- Common Uses:
 - Web Development
 - Data Science and Machine Learning
 - Automation and Scripting
 - Software Development
 - Artificial Intelligence
 - Scientific Computing

Installation Guide - Python

Python



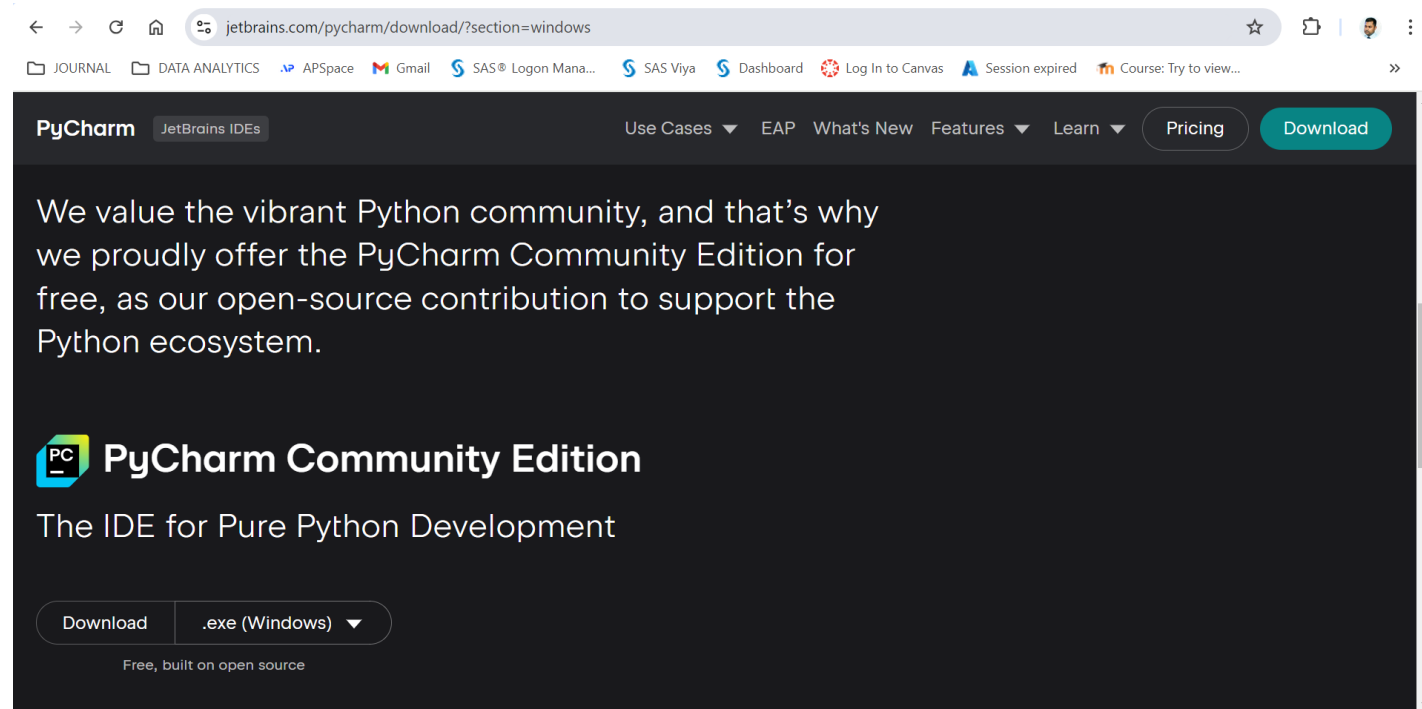
- Download and Install Python via
- <https://www.python.org/downloads/>
- Select the right version of Python according to the operating system of your laptop/desktop.
- Can use IDLE to write and execute python programs.
- Also, you can use either PyCharm or Visual Code as IDEs for your Python programming requirements.



Installation Guide - PyCharm



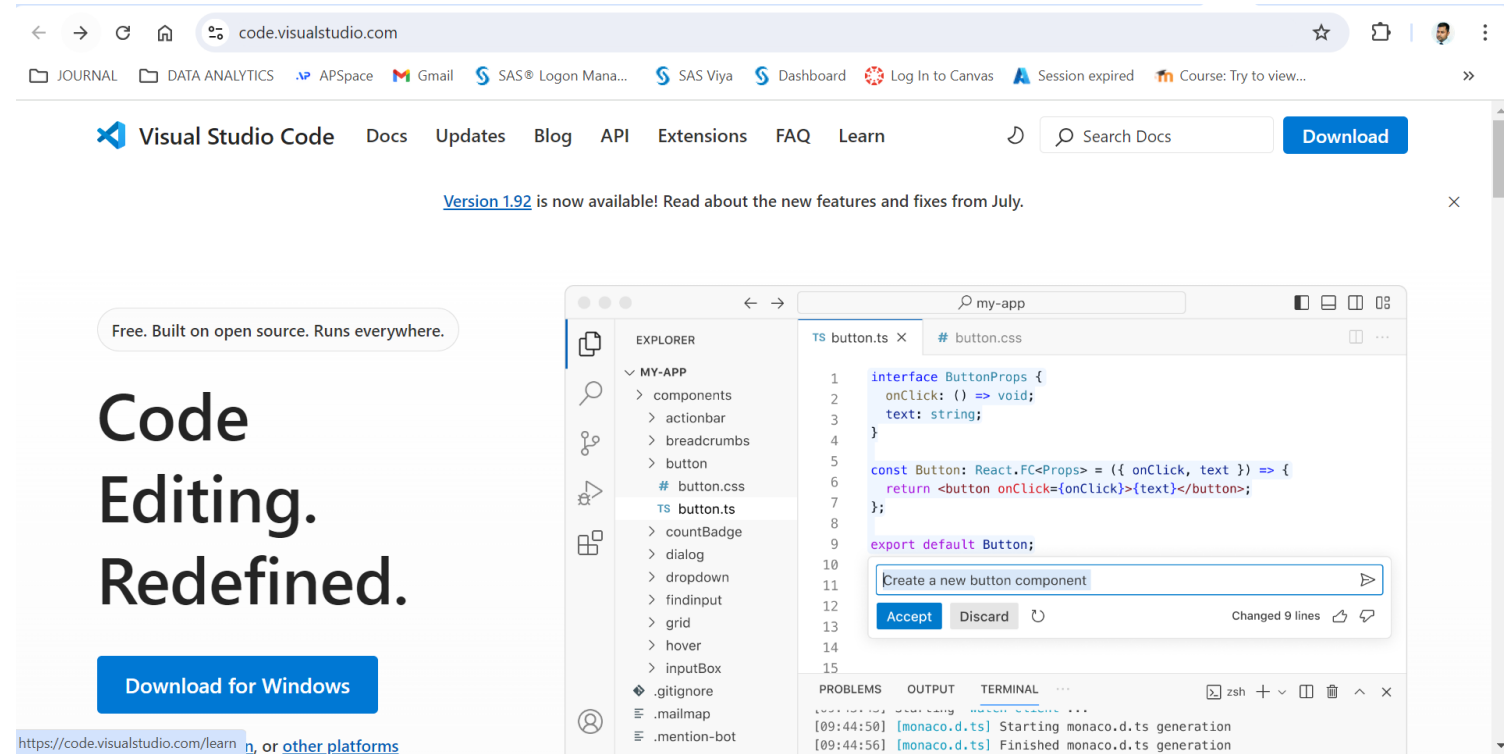
- Download and Install PyCharm via
- <https://www.jetbrains.com/pycharm/download/#section=windows>
- PyCharm Community Edition – Free IDE
- You may also get specific version according to the operating system you use via
- <https://www.jetbrains.com/pycharm/download/other.html>



Installation Guide - Visual Code

Visual Code

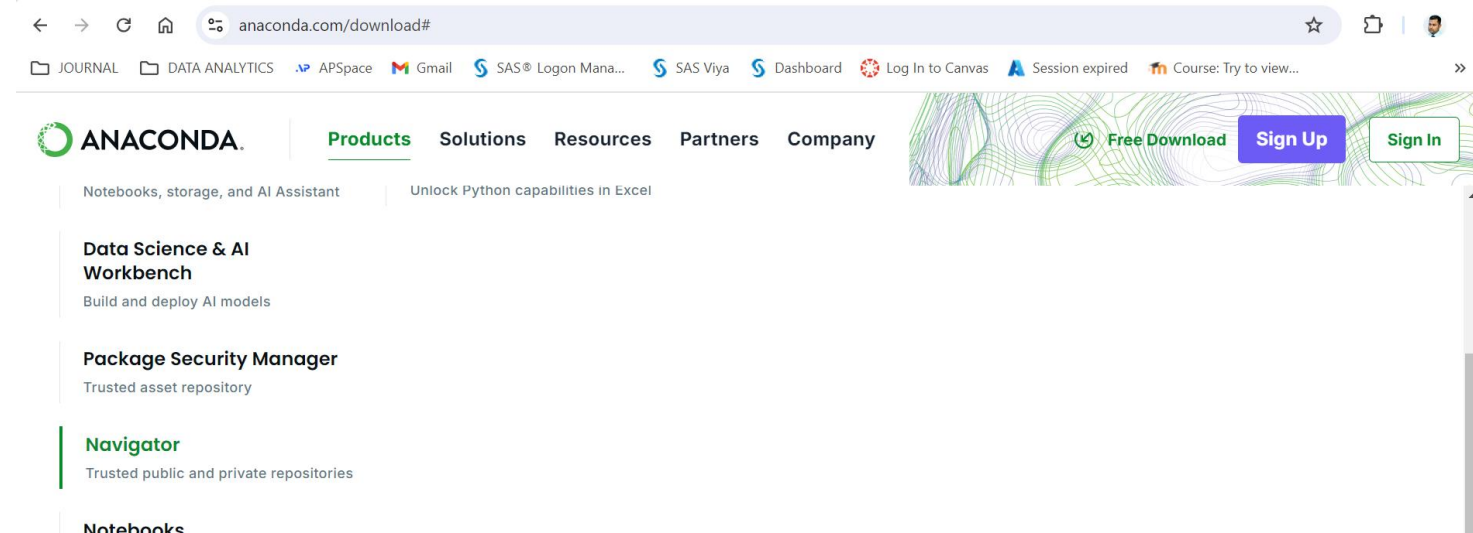
- Download and Install Visual Code via
- <https://code.visualstudio.com/>
- Select the specific one according to the operating system



Installation Guide - Anaconda



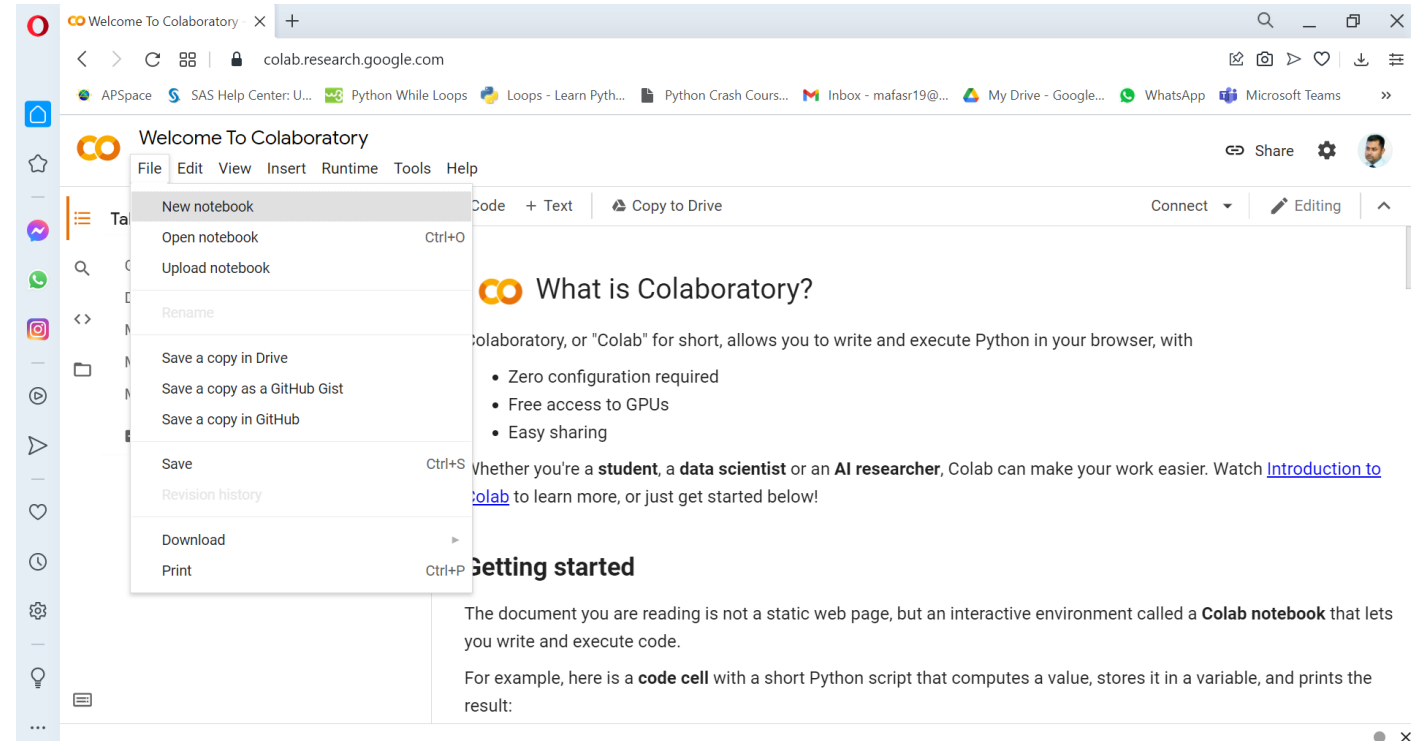
- Download and Install Anaconda via <https://www.anaconda.com/>
- Download Navigator and Install.
- After the installation, you will get an Anaconda Navigator and can load the Jupyter Lab or Jupyter Notebook to work with python programming.



Installation Guide - Colab

Colab 

- Use Colab via <https://colab.research.google.com>
- You need to have a Gmail account to use this facility.
- Colab is an IDE maintained by Google.

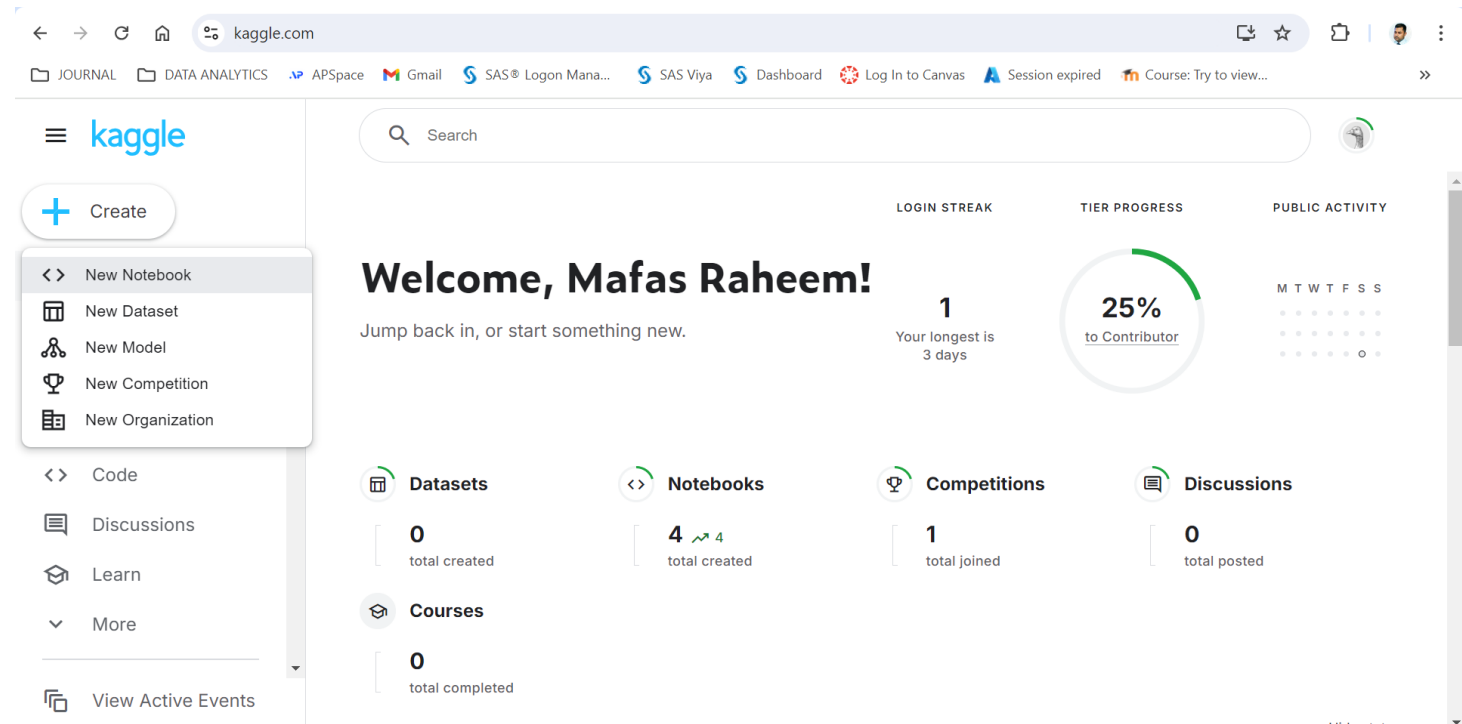


Installation Guide - Kaggle

Kaggle



- Use Kaggle via <https://www.kaggle.com/>
- You need to have an account to use this facility. Can use Gmail too.





Basics

Python Syntax vs. other programming languages

- Check the python version in Anaconda Jupyter or Colab

```
from platform import python_version  
python_version()
```

- A simple Syntax is followed when writing python codes.
- Indentation refers to the spaces at the beginning of a code line and is very important in python.
- Python uses indentation to indicate a block of code.
- Example:

```
print("Hello, World!")
```


Python Comments

- Comments are used to explain python code.
- It makes the code more readable.
- It prevents execution of code line(s) when testing code.

Example:

- `# This is a comment` - *Single line comment*
`print("Hello, World!")`

- Inline comment:
`print("Hello World")` *# This code prints "Hello World"*

- multiple line comments use `"""` or `'''` on either end.

`"""`

This type of comment spans multiple lines.

These are mostly used for documentation of functions, classes and modules. """

Variables

- Variables are data value storing containers in a programming language.
- In Python a variable is created when a value is assigned.

- **Example:**

```
a = 10
```

```
name = 'John' or name = "John"
```

- Can find the variable type/data type by using

```
type(name)
```

- Variable names are case-sensitive.

A variable name

- Must start with a letter or the underscore character.
- Cannot start with a number.
- Can only contain alpha-numeric characters and underscores (A-z, 0-9, and _).
- Is case-sensitive (name, Name and NAME are three different variables).

Variable – Input and Output

`print()` function → can be used for

single variable

```
a = "Python is awesome"  
print(a)
```

multiple variables

```
x = "Python"  
y = "is"  
z = "awesome"  
print(x, y, z)
```

```
a = input("Enter the value:")  
print(a)
```

Can use + operator too →

```
x = "Python "  
y = "is "  
z = "awesome"  
print(x + y + z)
```

Operator Precedence

Operators	Meaning
()	Parentheses
**	Exponent
+x, -x, ~x	Unary plus, Unary minus, Bitwise NOT
*, /, //, %	Multiplication, Division, Floor division, Modulus
+, -	Addition, Subtraction
<<, >>	Bitwise shift operators

Operator Precedence

Operators	Meaning
&	Bitwise AND
^	Bitwise XOR
	Bitwise OR
==, !=, >, >=, <, <=, is, is not, in, not in	Comparisons, Identity, Membership operators
not	Logical NOT
and	Logical AND
or	Logical OR

Collections in Python

List	Tuple	Set	Dictionary
ordered	ordered	unordered	ordered
changeable	unchangeable	unchangeable	changeable
Allows duplicate	Allows duplicate	No duplicate	No duplicate
		unindexed	

Collections in Python

LIST

- Example:
- `list = ["apple", "banana", "cherry"]`

TUPLE

- Example:
- `mytuple = ("apple", "banana", "cherry")`

SET

- Example:
- `my_set = {1, 2, 3, 4, 5}`

DICTIONARY

Example:

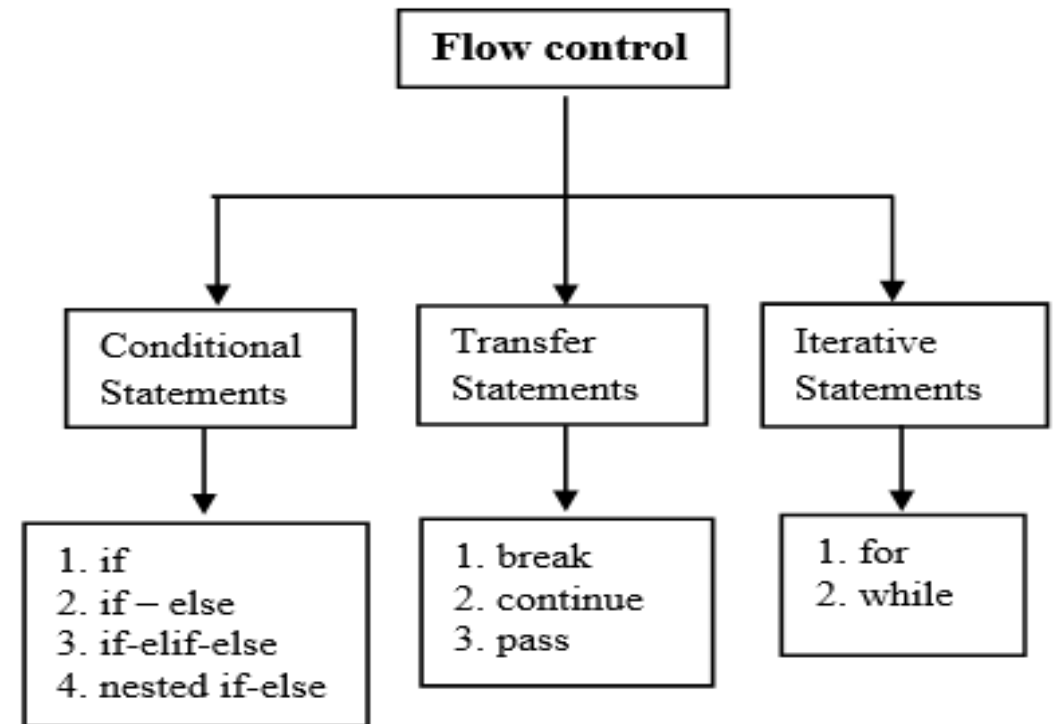
```
marks = {  
    "maths" : 86,  
    "science" : 82,  
    "english" : 93  
}
```

```
marks["maths"]
```

Control Flows in Python

The flow controls in python are divided into three types:

- Conditional statements
- Iterative statements.
- Transfer statements



Control Flows - Conditional statements

IF →

- Is the simplest form and evaluates a condition to either True or False.

Syntax:

if condition:

statement 1

statement n

Example:

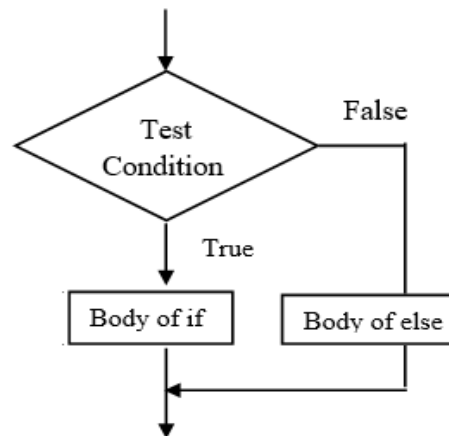
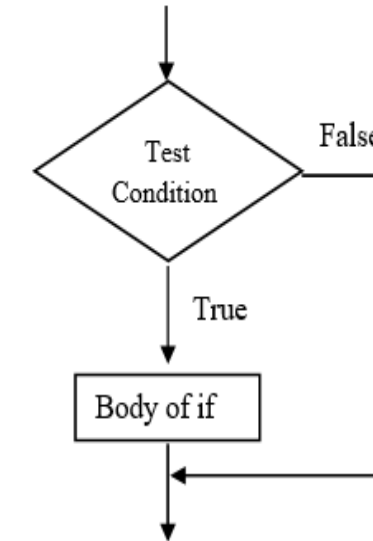
```
number = 6
```

```
if number > 5:
```

```
    # Calculate square
```

```
    print(number * number)
```

```
print('Next lines of code')
```



IF - ELSE →

Checks the condition and executes the if block of code when the condition is **True** and executes the else block of code if the condition is **False**.

Syntax:

if condition:

statement 1

else:

statement 2

Example:

```
password =
input('Enter password:
')
if password ==
"PYnative@#29":
    print("Correct
password")
else:
    print("Incorrect
Password")
```

Control Flows - Iterative Statements

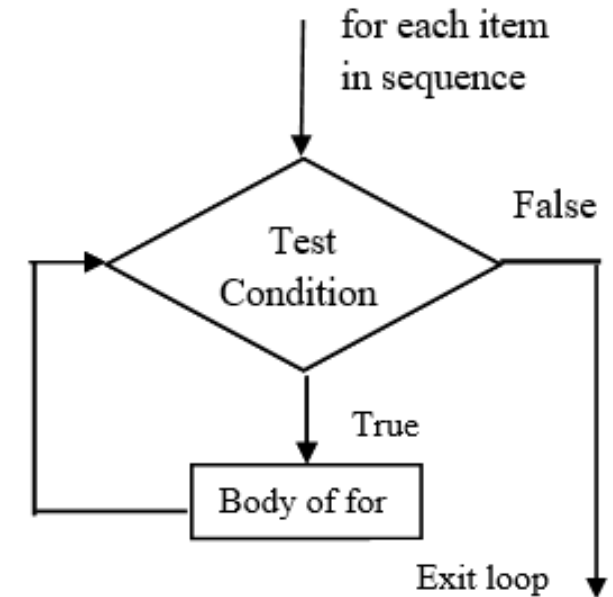
FOR loop

- Can iterate any sequence or iterable variable. The sequence can be string, list, dictionary, set, or tuple.
- Syntax:
- for i in range/sequences:
- statement 1
- statement 2
- statement n

Example:

```
for i in range(1, 11):  
    print(i)
```

```
~~~~~  
adj = ["red", "big", "tasty"]  
fruits = ["apple", "banana", "cherry"]  
for x in adj:  
    for y in fruits:  
        print(x, y)
```



Loop Through a List, Tuple, Dictionary and Tuple is possible

Control Flows - Iterative Statements

WHILE loop

Syntax

```
while condition:
    body of while loop
```

Repeatedly executes a code block while a particular condition is true.

Can also use ELSE

```
while condition: # until this condition is true
    statement 1
```

```
·
·
statement n
```

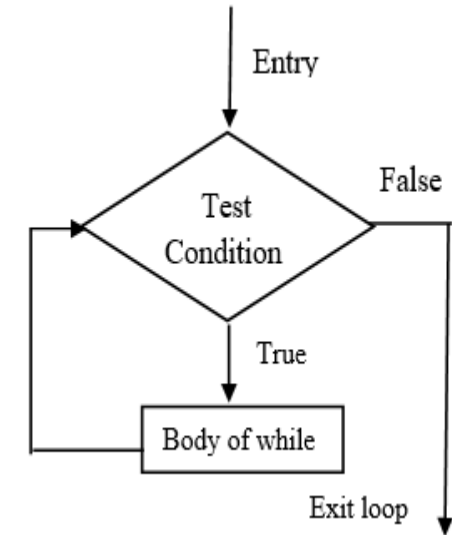
```
else:
    statement(s)
```

Example:

```
num = 10
sum = 0
i = 1
```

```
while i <= num:
    sum = sum + i
    i = i + 1
```

```
print("Sum of first 10 number is:", sum)
```



Body of while loop

Gets executed as long as the condition is true

Else block is optional

Gets executed when while loop is executed normally

User defined function in Python

A function is a block of code which runs when it is called. A function can accept parameters and arguments and can return value as a result.

In Python a function is defined using the **def** keyword:

```
def my_function():  
    print("My name is Mafas")  
my_function()
```

Example:

```
def my_function(name1, name2):  
    print("My name is", name1, name2)
```

```
my_function(name1 = "Mafas", name2 = "Raheem")
```

Example:

```
def my_function(name = "Mafas")  
:  
    print("My name is", name)
```

```
my_function()  
my_function("Aahil")
```




Libraries

Python Libraries

Required for Data Science Projects

- **Pandas** **pandas**

- Data Loading | Reading | Data Management | Basic Visualization

- **Matplotlib** **matplotlib**

- 2D Visualization

- **Seaborn** **seaborn**

- 2D & 3D Visualization

- **TensorFlow** **TensorFlow**

- Pre-processing | Data Split | Modelling | Evaluation

- **NumPy**



- 2D & 3D Visualization

- **Scikit Learn**



- Pre-processing | Data Split | Modelling | Evaluation

- **Keras** **Keras**

- Pre-processing | Data Split | Modelling | Evaluation





Pandas Library

Pandas

Data Management using Pandas



CREATE A DATAFRAME

```
import pandas as pd
df = pd.DataFrame({"Name" : ["Ann", "Barack", "Charles"],
                   "Age" : [25, 65, 70]}, index = [1, 2, 3])
df
```

LOAD A DATASET

```
import pandas as pd
data = pd.read_csv("C:/APU/MATERIALS/DATASETS/Churn_Modelling.csv")
data
```

Suitable read functions of Pandas library must be used to read the specific file format of the data.

Pandas

Data Management using Pandas



LOAD A DATASET

Load the data using the Pandas library. Pandas library can read/write the following data file formats.

Format Type	Data Description	Reader	Writer
text	CSV	read_csv	to_csv
text	Fixed-Width Text File	read_fwf	
text	JSON	read_json	to_json
text	HTML	read_html	to_html
text	LaTeX		Styler.to_latex
text	XML	read_xml	to_xml
text	Local clipboard	read_clipboard	to_clipboard
binary	MS Excel	read_excel	to_excel

Pandas

Data Management using Pandas



LOAD A DATASET

Load the data using the Pandas library. Pandas library can read/write the following data file formats.

binary	OpenDocument	read_excel	
binary	HDF5 Format	read_hdf	to_hdf
binary	Feather Format	read_feather	to_feather
binary	Parquet Format	read_parquet	to_parquet
binary	ORC Format	read_orc	
binary	Stata	read_stata	to_stata
binary	SAS	read_sas	
binary	SPSS	read_spss	
binary	Python Pickle Format	read_pickle	to_pickle

LOAD A DATASET

Load the data using the Pandas library. Pandas library can read/write the following data file formats.

```
data = pd.read_csv('file path')
```

```
data = pd.read_excel('file  
path', sheet_name = 'Sheet1')
```

SQL	SQL	read_sql	to_sql
SQL	Google BigQuery	read_gbq	to_gbq

VIEW DATA

```
# display the first 5 rows  
data.head()
```

```
# display the last 5 rows  
data.tail()
```

UNDERSTAND DATA

`data.shape` → dimension of the data

`len(data)` → display the no of records

`data.columns` → list out the columns

`data.dtypes` → column names and the respective data types

UNDERSTAND DATA

to get a summary of the categorical data

`data['Gender'].value_counts()` → any categorical variable can be used to get the details of it.

to get the mode

`data['Gender'].mode()`

to get the total number of records

`data['Gender'].count()`

UNDERSTAND DATA

to get the descriptive statistics

`data['EstimatedSalary'].describe()` → any numeric continuous variable can be used to get the details of it. The details will be:

count, mean, std, min, 25%, 50%, 75%, max

The following methods can also be exclusively used to get the details such as `count()`, `mean()`, `std()`, `min()`, `max()`

Pandas

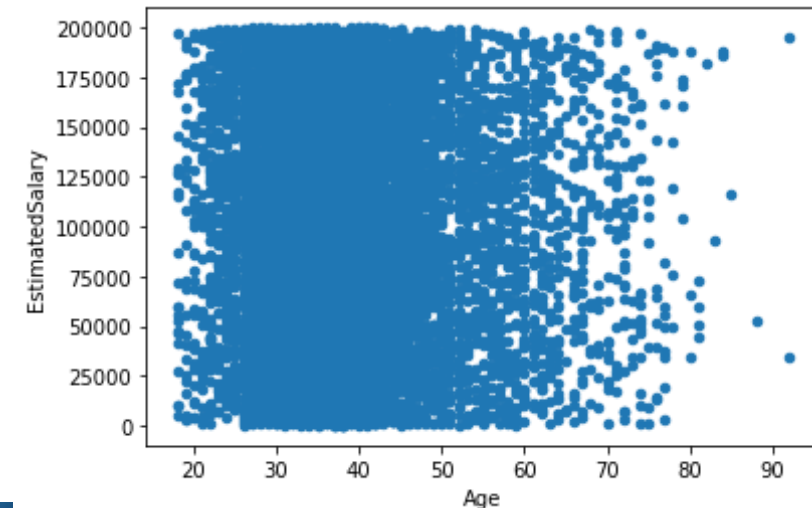
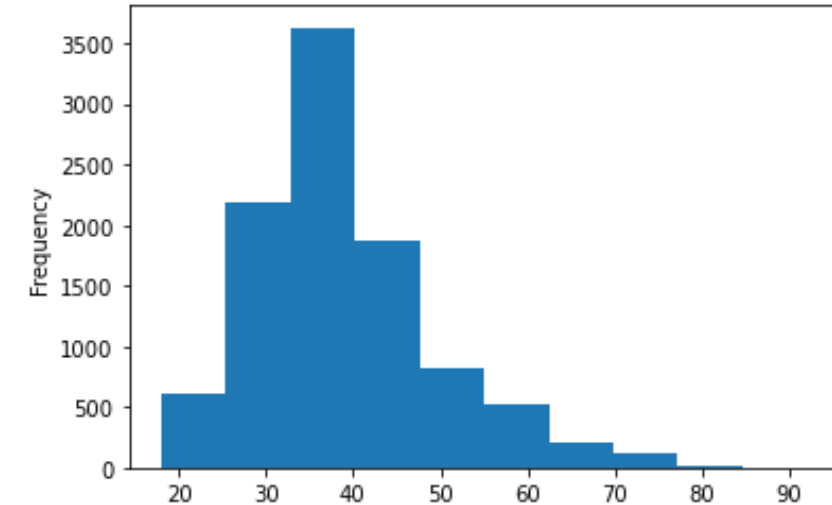
Data Management using Pandas

VISUALIZE DATA

to visualize → offered by PANDAS
library

```
data['Age'].plot.hist() # univariate
```

```
data.plot.scatter(x = 'Age', y =  
'EstimatedSalary') # bivariate
```



TAKE SAMPLE OF THE DATA

```
# take sample of the dataset
```

```
data_1 = data.sample(frac=0.5, random_state=1) # percentage to get  
the sample in a random manner
```

```
data_2 = data.sample(n=100, random_state=1)
```

```
# to export as csv
```

```
data_2.to_csv("C:/Users/raheem/OneDrive - Asia Pacific  
University/Desktop/data_2.csv")
```

MISSING DATA

```
# detect missing values  
data_new.isnull().sum()
```

```
# drop the rows which contain  
missing values  
data_new.dropna()
```

MANAGE DATA

```
# dropping unwanted variables  
= part of preprocessing
```

```
data_3 =  
data.drop('RowNumber',  
axis=1)
```

```
data_new =  
data.drop(data.columns[[0, 1,  
2]], axis=1)
```

MANAGE DATA

```
# convert the data type of a variable
```

```
data['Balance'] = data['Balance'].astype(int)
```

```
# create a new variable
```

```
data['Balance_new'] = data['Balance'].astype(int)
```

MANAGE DATA

group by → Descriptive analytics

```
data.groupby(['Geography', 'Gender'])['Geography'].count()
```

```
data.groupby(['Geography', 'Exited'])['Exited'].count()
```

```
data.groupby(['Geography', 'Age', 'Exited'])['Exited'].count()
```


MERGE DATA

- “Merging” is the process of bringing two datasets together into one and aligning the rows from each based on common attributes or columns.
- The words “merge” and “join” are used relatively interchangeably in Pandas and other languages and they both do the similar things.

pandas.DataFrame.merge

```
DataFrame.merge(right, how='inner', on=None, left_on=None, right_on=None,  
left_index=False, right_index=False, sort=False, suffixes=('_x', '_y'),  
copy=True, indicator=False, validate=None) [source]
```

Merge DataFrame or named Series objects with a database-style join.

MERGE DATA

```
import pandas as pd

data_1 = pd.DataFrame(
    {"X1" : ['A', 'B', 'C'],
     "X2" : [1, 2, 3]}
)
data_1
```

MERGE DATA

```
data_2 = pd.DataFrame(
    {"X1" : ['A', 'B', 'D'],
     "X3" : ['T', 'F', 'T']}
)
data_2

pd.merge(data_1 , data_2)
```

MERGE DATA

```
import pandas as pd
```

```
user_usage = pd.read_csv('C:/Users/raheem/OneDrive - Asia Pacific  
University/Desktop/Merge/user_usage.csv')
```

```
user_device = pd.read_csv('C:/Users/raheem/OneDrive - Asia Pacific  
University/Desktop/Merge/user_device.csv')
```

```
# identify a common variable among the datasets
```

```
result_1 = pd.merge(user_usage, user_device, on = 'use_id')
```

MERGE DATA

- Inner Merge
- Pandas uses “inner” merge by default. This keeps only the common values in both the left and right dataframes for the merged data.

```
# Inner Merge/Join
```

```
result_inner = pd.merge(user_usage, user_device[['use_id',  
'platform', 'device']], on = 'use_id', how = 'inner')
```

MERGE DATA

- Left Merge

- Keep every row in the left data frame. Where there are missing values of the “on” variable in the right data frame, add empty / NaN values in the result.

Left

```
result_left = pd.merge(user_usage, user_device[['use_id',  
'platform', 'device']], on = 'use_id', how = 'left')
```

MERGE DATA

- **Right Merge**
- Keep all the rows in the right data frame using the right parameter for the method how.

```
# Right
```

```
result_right = pd.merge(user_usage, user_device[['use_id',  
'platform', 'device']], on = 'use_id', how = 'right')
```

MERGE DATA

- **Outer Merge**
- The “outer” merge combines all the rows for left and right dataframes with NaN when there are no matched values in the rows.

Outer

```
result_outer = pd.merge(user_usage, user_device[['use_id',  
'platform', 'device']], on = 'use_id', how = 'outer')
```


Concatenating objects

- The `concat()` function (in the main pandas namespace) does all of the heavy lifting of performing concatenation operations along an axis while performing optional set logic (union or intersection) of the indexes (if any) on the other axes.
- `pd.concat(objs, axis=0, join="outer", ignore_index=False, keys=None, levels=None, names=None, verify_integrity=False, copy=True)`

<https://pandas.pydata.org/docs/reference/api/pandas.concat.html>

pd.concat vs pd.merge

- Concat function concatenates data frames along rows or columns. We can think of it as stacking up multiple data frames. Merge combines data frames based on values in shared columns. Merge function offers more flexibility compared to concat function because it allows combinations based on a condition.

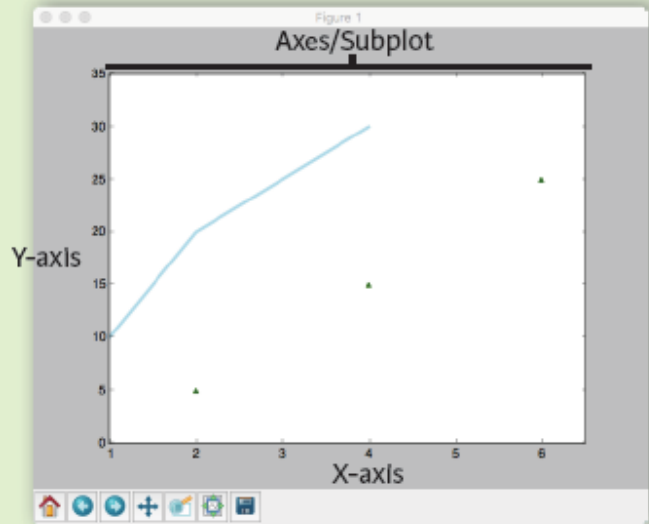
<https://pandas.pydata.org/docs/reference/api/pandas.concat.html>



Matplotlib and Seaborn Libraries

Plot Anatomy & Workflow

Plot Anatomy



Figure

Workflow

The basic steps to creating plots with matplotlib are:

- 1 Prepare data
- 2 Create plot
- 3 Plot
- 4 Customize plot
- 5 Save plot
- 6 Show plot

```
>>> import matplotlib.pyplot as plt
>>> x = [1,2,3,4]
>>> y = [10,20,25,30]
>>> fig = plt.figure()
>>> ax = fig.add_subplot(111)
>>> ax.plot(x, y, color='lightblue', linewidth=3)
>>> ax.scatter([2,4,6],
               [5,15,25],
               color='darkgreen',
               marker='^')
>>> ax.set_xlim(1, 6.5)
>>> plt.savefig('foo.png')
>>> plt.show()
```

- Matplotlib is a Python 2D plotting library which produces publication-quality figures in a variety of hardcopy formats and interactive environments across platforms.

2 Create Plot

```
>>> import matplotlib.pyplot as plt
```

Figure

```
>>> fig = plt.figure()  
>>> fig2 = plt.figure(figsize=plt.figaspect(2.0))
```

Axes

All plotting is done with respect to an `Axes`. In most cases, a subplot will fit your needs. A subplot is an axes on a grid system.

```
>>> fig.add_axes()  
>>> ax1 = fig.add_subplot(221) # row-col-num  
>>> ax3 = fig.add_subplot(212)  
>>> fig3, axes = plt.subplots(nrows=2,ncols=2)  
>>> fig4, axes2 = plt.subplots(ncols=3)
```


3 Plotting Routines

1D Data

<pre>>>> lines = ax.plot(x,y)</pre>	Draw points with lines or markers connecting them
<pre>>>> ax.scatter(x,y)</pre>	Draw unconnected points, scaled or colored
<pre>>>> axes[0,0].bar([1,2,3],[3,4,5])</pre>	Plot vertical rectangles (constant width)
<pre>>>> axes[1,0].barh([0.5,1,2.5],[0,1,2])</pre>	Plot horizontal rectangles (constant height)
<pre>>>> axes[1,1].axhline(0.45)</pre>	Draw a horizontal line across axes
<pre>>>> axes[0,1].axvline(0.65)</pre>	Draw a vertical line across axes
<pre>>>> ax.fill(x,y,color='blue')</pre>	Draw filled polygons
<pre>>>> ax.fill_between(x,y,color='yellow')</pre>	Fill between y-values and o

Vector Fields

<pre>>>> axes[0,1].arrow(0,0,0.5,0.5)</pre>	Add an arrow to the axes
<pre>>>> axes[1,1].quiver(y,z)</pre>	Plot a 2D field of arrows
<pre>>>> axes[0,1].streamplot(X,Y,U,V)</pre>	Plot 2D vector fields

Data Distributions

<pre>>>> ax1.hist(y)</pre>	Plot a histogram
<pre>>>> ax3.boxplot(y)</pre>	Make a box and whisker plot
<pre>>>> ax3.violinplot(z)</pre>	Make a violin plot

2D Data or Images

<pre>>>> fig, ax = plt.subplots() >>> im = ax.imshow(img, cmap='gist_earth', interpolation='nearest', vmin=-2, vmax=2)</pre>	Colormapped or RGB arrays	<pre>>>> axes2[0].pcolor(data2) >>> axes2[0].pcolormesh(data) >>> CS = plt.contour(Y,X,U) >>> axes2[2].contourf(data1) >>> axes2[2].ax.clabel(CS)</pre>	Pseudocolor plot of 2D array Pseudocolor plot of 2D array Plot contours Plot filled contours Label a contour plot
--	---------------------------	--	---

4 Customize Plot

Colors, Color Bars & Color Maps

```
>>> plt.plot(x, x, x, x**2, x, x**3)
>>> ax.plot(x, y, alpha = 0.4)
>>> ax.plot(x, y, c='k')
>>> fig.colorbar(im, orientation='horizontal')
>>> im = ax.imshow(img,
                    cmap='seismic')
```

Markers

```
>>> fig, ax = plt.subplots()
>>> ax.scatter(x, y, marker=".")
>>> ax.plot(x, y, marker="o")
```

Linestyles

```
>>> plt.plot(x, y, linewidth=4.0)
>>> plt.plot(x, y, ls='solid')
>>> plt.plot(x, y, ls='--')
>>> plt.plot(x, y, '--', x**2, y**2, '-.')
>>> plt.setp(lines, color='r', linewidth=4.0)
```

Text & Annotations

```
>>> ax.text(1,
          -2.1,
          'Example Graph',
          style='italic')
>>> ax.annotate("Sine",
               xy=(8, 0),
               xycoords='data',
               xytext=(10.5, 0),
               textcoords='data',
               arrowprops=dict(arrowstyle="->",
                               connectionstyle="arc3"),)
```

Mattext

```
>>> plt.title(r'$\sigma_i=15$', fontsize=20)
```

Limits, Legends & Layouts

Limits & Autoscaling

```
>>> ax.margins(x=0.0, y=0.1)
>>> ax.axis('equal')
>>> ax.set(xlim=[0, 10.5], ylim=[-1.5, 1.5])
>>> ax.set_xlim(0, 10.5)
```

Legends

```
>>> ax.set(title='An Example Axes',
          ylabel='Y-Axis',
          xlabel='X-Axis')
>>> ax.legend(loc='best')
```

Ticks

```
>>> ax.xaxis.set(ticks=range(1, 5),
                ticklabels=[3, 100, -12, "foo"])
>>> ax.tick_params(axis='y',
                  direction='inout',
                  length=10)
```

Subplot Spacing

```
>>> fig3.subplots_adjust(wspace=0.5,
                        hspace=0.3,
                        left=0.125,
                        right=0.9,
                        top=0.9,
                        bottom=0.1)
>>> fig.tight_layout()
```

Axis Spines

```
>>> ax1.spines['top'].set_visible(False)
>>> ax1.spines['bottom'].set_position(('outward', 10))
```

Add padding to a plot
Set the aspect ratio of the plot to 1
Set limits for x-and y-axis
Set limits for x-axis

Set a title and x-and y-axis labels

No overlapping plot elements

Manually set x-ticks

Make y-ticks longer and go in and out

Adjust the spacing between subplots

Fit subplot(s) in to the figure area

Make the top axis line for a plot invisible
Move the bottom axis line outward

5 Save Plot

Save figures

```
>>> plt.savefig('foo.png')
```

Save transparent figures

```
>>> plt.savefig('foo.png', transparent=True)
```

6 Show Plot

```
>>> plt.show()
```

Close & Clear

```
>>> plt.cla()
```

```
>>> plt.clf()
```

```
>>> plt.close()
```

Clear an axis

Clear the entire figure

Close a window

Statistical Data Visualization With Seaborn

The Python visualization library Seaborn is based on matplotlib and provides a high-level interface for drawing attractive statistical graphics.

Make use of the following aliases to import the libraries:

```
>>> import matplotlib.pyplot as plt
>>> import seaborn as sns
```

The basic steps to creating plots with Seaborn are:

1. Prepare some data
2. Control figure aesthetics
3. Plot with Seaborn
4. Further customize your plot

```
>>> import matplotlib.pyplot as plt
>>> import seaborn as sns
>>> tips = sns.load_dataset("tips")
>>> sns.set_style("whitegrid")
>>> g = sns.lmplot(x="tip",
                  y="total_bill",
                  data=tips,
                  aspect=2)
>>> g = (g.set_axis_labels("Tip", "Total bill (USD)").
        set(xlim=(0,10), ylim=(0,100)))
>>> plt.title("title")
>>> plt.show(g)
```

1 Data

Also see Lists, NumPy & Pandas

```
>>> import pandas as pd
>>> import numpy as np
>>> uniform_data = np.random.rand(10, 12)
>>> data = pd.DataFrame({'x':np.arange(1,101),
                        'y':np.random.normal(0,4,100)})
```

Seaborn also offers built-in data sets:

```
>>> titanic = sns.load_dataset("titanic")
>>> iris = sns.load_dataset("iris")
```

2 Figure Aesthetics Also see Matplotlib

```
>>> f, ax = plt.subplots(figsize=(5,6))
```

Create a figure and one subplot

Seaborn styles

```
>>> sns.set()
>>> sns.set_style("whitegrid")
>>> sns.set_style("ticks",
                    {"xtick.major.size":8,
                     "ytick.major.size":8})
>>> sns.axes_style("whitegrid")
```

(Re)set the seaborn default
Set the matplotlib parameters
Set the matplotlib parameters

Return a dict of params or use with
with to temporarily set the style

Context Functions

```
>>> sns.set_context("talk")
>>> sns.set_context("notebook",
                    font_scale=1.5,
                    rc={"lines.linewidth":2.5})
```

Set context to "talk"
Set context to "notebook",
scale font elements and
override param mapping

Color Palette

```
>>> sns.set_palette("husl",3)
>>> sns.color_palette("husl")
>>> flatui = ["#9b59b6", "#3498db", "#95a5a6", "#e74c3c", "#34495e", "#2ecc71"]
>>> sns.set_palette(flatui)
```

Define the color palette
Use with with to temporarily set palette
Set your own color palette

3 Plotting With Seaborn

Axis Grids

```
>>> g = sns.FacetGrid(titanic,
                      col="survived",
                      row="sex")
>>> g = g.map(plt.hist, "age")
>>> sns.factorplot(x="pclass",
                  y="survived",
                  hue="sex",
                  data=titanic)
>>> sns.lmplot(x="sepal_width",
              y="sepal_length",
              hue="species",
              data=iris)
```

Subplot grid for plotting conditional relationships

Draw a categorical plot onto a Facetgrid

Plot data and regression model fits across a FacetGrid

```
>>> h = sns.PairGrid(iris)
>>> h = h.map(plt.scatter)
>>> sns.pairplot(iris)
>>> i = sns.JointGrid(x="x",
                     y="y",
                     data=data)
>>> i = i.plot(sns.regplot,
               sns.distplot)
>>> sns.jointplot("sepal_length",
                 "sepal_width",
                 data=iris,
                 kind='kde')
```

Subplot grid for plotting pairwise relationships
Plot pairwise bivariate distributions
Grid for bivariate plot with marginal univariate plots

Plot bivariate distribution

Categorical Plots

Scatterplot

```
>>> sns.stripplot(x="species",
                  y="petal_length",
                  data=iris)
>>> sns.swarmplot(x="species",
                  y="petal_length",
                  data=iris)
```

Scatterplot with one categorical variable

Categorical scatterplot with non-overlapping points

Bar Chart

```
>>> sns.barplot(x="sex",
                y="survived",
                hue="class",
                data=titanic)
```

Show point estimates and confidence intervals with scatterplot glyphs

Count Plot

```
>>> sns.countplot(x="deck",
                  data=titanic,
                  palette="Greens_d")
```

Show count of observations

Point Plot

```
>>> sns.pointplot(x="class",
                  y="survived",
                  hue="sex",
                  data=titanic,
                  palette={"male": "g",
                           "female": "m"},
                  markers=["^", "o"],
                  linestyle=["-", "--"])
```

Show point estimates and confidence intervals as rectangular bars

Boxplot

```
>>> sns.boxplot(x="alive",
                y="age",
                hue="adult_male",
                data=titanic)
>>> sns.boxplot(data=iris, orient="h")
```

Boxplot

Boxplot with wide-form data

Violinplot

```
>>> sns.violinplot(x="age",
                   y="sex",
                   hue="survived",
                   data=titanic)
```

Violin plot

Regression Plots

```
>>> sns.regplot(x="sepal_width",
                 y="sepal_length",
                 data=iris,
                 ax=ax)
```

Plot data and a linear regression model fit

Distribution Plots

```
>>> plot = sns.distplot(data.y,
                        kde=False,
                        color="b")
```

Plot univariate distribution

Matrix Plots

```
>>> sns.heatmap(uniform_data, vmin=0, vmax=1)
```

Heatmap

4 Further Customizations

Also see Matplotlib

Axisgrid Objects

<code>>>> g.despine(left=True)</code>	Remove left spine
<code>>>> g.set_ylabels("Survived")</code>	Set the labels of the y-axis
<code>>>> g.set_xticklabels(rotation=45)</code>	Set the tick labels for x
<code>>>> g.set_axis_labels("Survived", "Sex")</code>	Set the axis labels
<code>>>> h.set(xlim=(0,5), ylim=(0,5), xticks=[0,2.5,5], yticks=[0,2.5,5])</code>	Set the limit and ticks of the x-and y-axis

Plot

<code>>>> plt.title("A Title")</code>	Add plot title
<code>>>> plt.ylabel("Survived")</code>	Adjust the label of the y-axis
<code>>>> plt.xlabel("Sex")</code>	Adjust the label of the x-axis
<code>>>> plt.ylim(0,100)</code>	Adjust the limits of the y-axis
<code>>>> plt.xlim(0,10)</code>	Adjust the limits of the x-axis
<code>>>> plt.setp(ax, yticks=[0,5])</code>	Adjust a plot property
<code>>>> plt.tight_layout()</code>	Adjust subplot params

5 Show or Save Plot

Also see Matplotlib

<code>>>> plt.show()</code>	Show the plot
<code>>>> plt.savefig("foo.png")</code>	Save the plot as a figure
<code>>>> plt.savefig("foo.png", transparent=True)</code>	Save transparent figure

Close & Clear

Also see Matplotlib

<code>>>> plt.cla()</code>	Clear an axis
<code>>>> plt.clf()</code>	Clear an entire figure
<code>>>> plt.close()</code>	Close a window



Scikit Learn Library

Scikit-learn

Scikit-learn is an open source Python library that implements a range of machine learning, preprocessing, cross-validation and visualization algorithms using a unified interface.



A Basic Example

```
>>> from sklearn import neighbors, datasets, preprocessing
>>> from sklearn.model_selection import train_test_split
>>> from sklearn.metrics import accuracy_score
>>> iris = datasets.load_iris()
>>> X, y = iris.data[:, :2], iris.target
>>> X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=33)
>>> scaler = preprocessing.StandardScaler().fit(X_train)
>>> X_train = scaler.transform(X_train)
>>> X_test = scaler.transform(X_test)
>>> knn = neighbors.KNeighborsClassifier(n_neighbors=5)
>>> knn.fit(X_train, y_train)
>>> y_pred = knn.predict(X_test)
>>> accuracy_score(y_test, y_pred)
```

Loading The Data

Also see NumPy & Pandas

Your data needs to be numeric and stored as NumPy arrays or SciPy sparse matrices. Other types that are convertible to numeric arrays, such as Pandas DataFrame, are also acceptable.

```
>>> import numpy as np
>>> X = np.random.random((10,5))
>>> y = np.array(['M', 'M', 'F', 'F', 'M', 'F', 'M', 'M', 'F', 'F', 'F'])
>>> X[X < 0.7] = 0
```

Training And Test Data

```
>>> from sklearn.model_selection import train_test_split
>>> X_train, X_test, y_train, y_test = train_test_split(X,
                                                         y,
                                                         random_state=0)
```


Preprocessing The Data

Standardization

```
>>> from sklearn.preprocessing import StandardScaler
>>> scaler = StandardScaler().fit(X_train)
>>> standardized_X = scaler.transform(X_train)
>>> standardized_X_test = scaler.transform(X_test)
```

Normalization

```
>>> from sklearn.preprocessing import Normalizer
>>> scaler = Normalizer().fit(X_train)
>>> normalized_X = scaler.transform(X_train)
>>> normalized_X_test = scaler.transform(X_test)
```

Binarization

```
>>> from sklearn.preprocessing import Binarizer
>>> binarizer = Binarizer(threshold=0.0).fit(X)
>>> binary_X = binarizer.transform(X)
```

Encoding Categorical Features

```
>>> from sklearn.preprocessing import LabelEncoder
>>> enc = LabelEncoder()
>>> y = enc.fit_transform(y)
```

Imputing Missing Values

```
>>> from sklearn.preprocessing import Imputer
>>> imp = Imputer(missing_values=0, strategy='mean', axis=0)
>>> imp.fit_transform(X_train)
```

Generating Polynomial Features

```
>>> from sklearn.preprocessing import PolynomialFeatures
>>> poly = PolynomialFeatures(5)
>>> poly.fit_transform(X)
```

Create Your Model

Supervised Learning Estimators

Linear Regression

```
>>> from sklearn.linear_model import LinearRegression
>>> lr = LinearRegression(normalize=True)
```

Support Vector Machines (SVM)

```
>>> from sklearn.svm import SVC
>>> svc = SVC(kernel='linear')
```

Naive Bayes

```
>>> from sklearn.naive_bayes import GaussianNB
>>> gnb = GaussianNB()
```

KNN

```
>>> from sklearn import neighbors
>>> knn = neighbors.KNeighborsClassifier(n_neighbors=5)
```

Unsupervised Learning Estimators

Principal Component Analysis (PCA)

```
>>> from sklearn.decomposition import PCA
>>> pca = PCA(n_components=0.95)
```

K Means

```
>>> from sklearn.cluster import KMeans
>>> k_means = KMeans(n_clusters=3, random_state=0)
```

Model Fitting

Supervised learning

```
>>> lr.fit(X, y)
>>> knn.fit(X_train, y_train)
>>> svc.fit(X_train, y_train)
```

Fit the model to the data

Unsupervised Learning

```
>>> k_means.fit(X_train)
>>> pca_model = pca.fit_transform(X_train)
```

Fit the model to the data
Fit to data, then transform it

Prediction

Supervised Estimators

```
>>> y_pred = svc.predict(np.random.random((2,5)))
>>> y_pred = lr.predict(X_test)
>>> y_pred = knn.predict_proba(X_test)
```

Predict labels
Predict labels
Estimate probability of a label

Unsupervised Estimators

```
>>> y_pred = k_means.predict(X_test)
```

Predict labels in clustering algos

Classification Metrics

Accuracy Score

```
>>> knn.score(X_test, y_test)
```

Estimator score method

```
>>> from sklearn.metrics import accuracy_score
```

Metric scoring functions

```
>>> accuracy_score(y_test, y_pred)
```

Classification Report

```
>>> from sklearn.metrics import classification_report
```

Precision, recall, f1-score
and support

```
>>> print(classification_report(y_test, y_pred))
```

Confusion Matrix

```
>>> from sklearn.metrics import confusion_matrix
```

```
>>> print(confusion_matrix(y_test, y_pred))
```

Regression Metrics

Mean Absolute Error

```
>>> from sklearn.metrics import mean_absolute_error
```

```
>>> y_true = [3, -0.5, 2]
```

```
>>> mean_absolute_error(y_true, y_pred)
```

Mean Squared Error

```
>>> from sklearn.metrics import mean_squared_error
```

```
>>> mean_squared_error(y_test, y_pred)
```

R² Score

```
>>> from sklearn.metrics import r2_score
```

```
>>> r2_score(y_true, y_pred)
```

Clustering Metrics

Adjusted Rand Index

```
>>> from sklearn.metrics import adjusted_rand_score
```

```
>>> adjusted_rand_score(y_true, y_pred)
```

Homogeneity

```
>>> from sklearn.metrics import homogeneity_score
```

```
>>> homogeneity_score(y_true, y_pred)
```

V-measure

```
>>> from sklearn.metrics import v_measure_score
```

```
>>> metrics.v_measure_score(y_true, y_pred)
```

Cross-Validation

```
>>> from sklearn.cross_validation import cross_val_score
```

```
>>> print(cross_val_score(knn, X_train, y_train, cv=4))
```

```
>>> print(cross_val_score(lr, X, y, cv=2))
```

Tune Your Model

Grid Search

```
>>> from sklearn.grid_search import GridSearchCV
>>> params = {"n_neighbors": np.arange(1,3),
              "metric": ["euclidean", "cityblock"]}
>>> grid = GridSearchCV(estimator=knn,
                        param_grid=params)
>>> grid.fit(X_train, y_train)
>>> print(grid.best_score_)
>>> print(grid.best_estimator_.n_neighbors)
```

Randomized Parameter Optimization

```
>>> from sklearn.grid_search import RandomizedSearchCV
>>> params = {"n_neighbors": range(1,5),
              "weights": ["uniform", "distance"]}
>>> rsearch = RandomizedSearchCV(estimator=knn,
                                 param_distributions=params,
                                 cv=4,
                                 n_iter=8,
                                 random_state=5)
>>> rsearch.fit(X_train, y_train)
>>> print(rsearch.best_score_)
```




TensorFlow – Keras Library

Keras

Keras is a powerful and easy-to-use deep learning library for Theano and TensorFlow that provides a high-level neural networks API to develop and evaluate deep learning models.

A Basic Example

```
>>> import numpy as np
>>> from keras.models import Sequential
>>> from keras.layers import Dense
>>> data = np.random.random((1000,100))
>>> labels = np.random.randint(2,size=(1000,1))
>>> model = Sequential()
>>> model.add(Dense(32,
                    activation='relu',
                    input_dim=100))
>>> model.add(Dense(1, activation='sigmoid'))
>>> model.compile(optimizer='rmsprop',
                  loss='binary_crossentropy',
                  metrics=['accuracy'])
>>> model.fit(data,labels,epochs=10,batch_size=32)
>>> predictions = model.predict(data)
```

Data

Also see NumPy, Pandas & Scikit-Learn

Your data needs to be stored as NumPy arrays or as a list of NumPy arrays. Ideally, you split the data in training and test sets, for which you can also resort to the `train_test_split` module of `sklearn.cross_validation`.

Keras Data Sets

```
>>> from keras.datasets import boston_housing,
                                mnist,
                                cifar10,
                                imdb
>>> (x_train,y_train),(x_test,y_test) = mnist.load_data()
>>> (x_train2,y_train2),(x_test2,y_test2) = boston_housing.load_data()
>>> (x_train3,y_train3),(x_test3,y_test3) = cifar10.load_data()
>>> (x_train4,y_train4),(x_test4,y_test4) = imdb.load_data(num_words=20000)
>>> num_classes = 10
```

Other

```
>>> from urllib.request import urlopen
>>> data = np.loadtxt(urlopen("http://archive.ics.uci.edu/
ml/machine-learning-databases/pima-indians-diabetes/
pima-indians-diabetes.data"),delimiter=",")
>>> X = data[:,0:8]
>>> y = data[:,8]
```

Preprocessing

Also see NumPy & Scikit-Learn

Sequence Padding

```
>>> from keras.preprocessing import sequence
>>> x_train4 = sequence.pad_sequences(x_train4,maxlen=80)
>>> x_test4 = sequence.pad_sequences(x_test4,maxlen=80)
```

One-Hot Encoding

```
>>> from keras.utils import to_categorical
>>> Y_train = to_categorical(y_train, num_classes)
>>> Y_test = to_categorical(y_test, num_classes)
>>> Y_train3 = to_categorical(y_train3, num_classes)
>>> Y_test3 = to_categorical(y_test3, num_classes)
```

Train and Test Sets

```
>>> from sklearn.model_selection import train_test_split
>>> X_train5,X_test5,y_train5,y_test5 = train_test_split(X,
                                                         y,
                                                         test_size=0.33,
                                                         random_state=42)
```

Standardization/Normalization

```
>>> from sklearn.preprocessing import StandardScaler
>>> scaler = StandardScaler().fit(x_train2)
>>> standardized_X = scaler.transform(x_train2)
>>> standardized_X_test = scaler.transform(x_test2)
```


TensorFlow - Keras



Sequential Model

```
>>> from keras.models import Sequential
>>> model = Sequential()
>>> model2 = Sequential()
>>> model3 = Sequential()
```

Multilayer Perceptron (MLP)

Binary Classification

```
>>> from keras.layers import Dense
>>> model.add(Dense(12,
                    input_dim=8,
                    kernel_initializer='uniform',
                    activation='relu'))
>>> model.add(Dense(8, kernel_initializer='uniform', activation='relu'))
>>> model.add(Dense(1, kernel_initializer='uniform', activation='sigmoid'))
```

Multi-Class Classification

```
>>> from keras.layers import Dropout
>>> model.add(Dense(512, activation='relu', input_shape=(784,)))
>>> model.add(Dropout(0.2))
>>> model.add(Dense(512, activation='relu'))
>>> model.add(Dropout(0.2))
>>> model.add(Dense(10, activation='softmax'))
```

Regression

```
>>> model.add(Dense(64, activation='relu', input_dim=train_data.shape[1]))
>>> model.add(Dense(1))
```

Convolutional Neural Network (CNN)

```
>>> from keras.layers import Activation, Conv2D, MaxPooling2D, Flatten
>>> model2.add(Conv2D(32, (3, 3), padding='same', input_shape=x_train.shape[1:]))
>>> model2.add(Activation('relu'))
>>> model2.add(Conv2D(32, (3, 3)))
>>> model2.add(Activation('relu'))
>>> model2.add(MaxPooling2D(pool_size=(2, 2)))
>>> model2.add(Dropout(0.25))
>>> model2.add(Conv2D(64, (3, 3), padding='same'))
>>> model2.add(Activation('relu'))
>>> model2.add(Conv2D(64, (3, 3)))
>>> model2.add(Activation('relu'))
>>> model2.add(MaxPooling2D(pool_size=(2, 2)))
>>> model2.add(Dropout(0.25))
>>> model2.add(Flatten())
>>> model2.add(Dense(512))
>>> model2.add(Activation('relu'))
>>> model2.add(Dropout(0.5))
>>> model2.add(Dense(num_classes))
>>> model2.add(Activation('softmax'))
```

Recurrent Neural Network (RNN)

```
>>> from keras.layers import Embedding, LSTM
>>> model3.add(Embedding(20000, 128))
>>> model3.add(LSTM(128, dropout=0.2, recurrent_dropout=0.2))
>>> model3.add(Dense(1, activation='sigmoid'))
```

TensorFlow - Keras



Inspect Model

<pre>>>> model.output_shape</pre>	Model output shape
<pre>>>> model.summary()</pre>	Model summary representation
<pre>>>> model.get_config()</pre>	Model configuration
<pre>>>> model.get_weights()</pre>	List all weight tensors in the model

Compile Model

MLP: Binary Classification

```
>>> model.compile(optimizer='adam',  
                  loss='binary_crossentropy',  
                  metrics=['accuracy'])
```

MLP: Multi-Class Classification

```
>>> model.compile(optimizer='rmsprop',  
                  loss='categorical_crossentropy',  
                  metrics=['accuracy'])
```

MLP: Regression

```
>>> model.compile(optimizer='rmsprop',  
                  loss='mse',  
                  metrics=['mae'])
```

Recurrent Neural Network

```
>>> model3.compile(loss='binary_crossentropy',  
                   optimizer='adam',  
                   metrics=['accuracy'])
```

Model Training

```
>>> model3.fit(x_train4,  
               y_train4,  
               batch_size=32,  
               epochs=15,  
               verbose=1,  
               validation_data=(x_test4,y_test4))
```

Evaluate Your Model's Performance

```
>>> score = model3.evaluate(x_test,  
                            y_test,  
                            batch_size=32)
```

Prediction

```
>>> model3.predict(x_test4, batch_size=32)  
>>> model3.predict_classes(x_test4,batch_size=32)
```

Save/Reload Models

```
>>> from keras.models import load_model
>>> model3.save('model_file.h5')
>>> my_model = load_model('my_model.h5')
```

Model Fine-tuning

Optimization Parameters

```
>>> from keras.optimizers import RMSprop
>>> opt = RMSprop(lr=0.0001, decay=1e-6)
>>> model2.compile(loss='categorical_crossentropy',
                    optimizer=opt,
                    metrics=['accuracy'])
```

Early Stopping

```
>>> from keras.callbacks import EarlyStopping
>>> early_stopping_monitor = EarlyStopping(patience=2)
>>> model3.fit(x_train4,
              y_train4,
              batch_size=32,
              epochs=15,
              validation_data=(x_test4, y_test4),
              callbacks=[early_stopping_monitor])
```

Review Questions

- What are python-based libraries suitable for a data science project?
- Explore different functions to handle dataset.
- Explore the functions available for data preprocessing.

Summary / Recap of Main Points

- Python is a widely used programming language for data science.
- It is Readable and Simple Syntax, Interpreted Language, Dynamically Typed, Extensive Standard Library, Cross-Platform, and Open-Source programming language.
- Various IDEs available.
- Wide range of libraries available for an effective data science project.

What To Expect Next Week

In Class

- Introduction to Supervised Learning

Preparation for Class

- Introduction to Supervised Learning